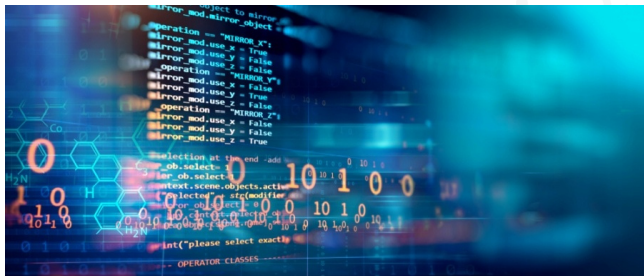


Problemas de Programación Funcional

Tecnología de Programación



Adolfo Muñoz – Juan Magallón
Grado en Ingeniería Informática



Universidad
Zaragoza



Escuela de
Ingeniería y Arquitectura
Universidad Zaragoza



Departamento de
Informática e Ingeniería
de Sistemas
Universidad Zaragoza

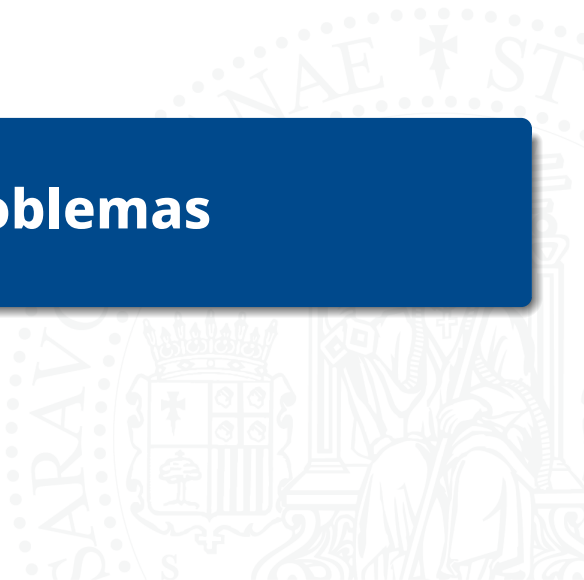
Objetivo

Objetivo

A continuación se plantearán varios problemas. Para cada uno de ellos:

- Verás una breve descripción de las funciones de Haskell que puede que necesites conocer.
- Verás la descripción del problema.
- Intenta resolver el problema de distintas formas:
 - usando sólo recursividad y ajuste de patrones.
 - usando las funciones disponibles en Haskell.
- Intenta reducir la expresión de la función mediante la notación *point-free* y el operador de composición de funciones (`.`).

Problemas

A faint, light gray watermark of the University of Seville seal is visible in the background. The seal is circular and features a central shield with various heraldic symbols, including a cross and a tree. The shield is topped with a crown. The words "UNIVERSITAS" and "SEVILLAE" are inscribed around the perimeter of the seal.

Funciones Haskell

- `map :: (a -> b) -> [a] -> [b]`
Devuelve la lista resultante de aplicar la función a cada uno de los elementos de la lista de entrada.
- `sum :: Num a => [a] -> a`
Devuelve la suma de todos los elementos de la lista.
- `zip :: [a] -> [b] -> [(a, b)]`
Devuelve una lista de tuplas, cada una de ellas conteniendo un elemento de cada una de las listas parámetro en sus correspondientes posiciones.
- `zipWith :: (a -> b -> c) -> [a] -> [b] -> [c]`
Devuelve una lista resultado de aplicar la función binaria a cada par de elementos (uno de cada lista) en sus correspondientes posiciones.

Problema 1

Define una función que calcule el producto escalar de dos vectores, representados mediante listas.

```
1 dot :: Num a => [a] -> [a] -> a
```

Funciones Haskell

- `map :: (a -> b) -> [a] -> [b]`

Devuelve la lista resultante de aplicar la función a cada uno de los elementos de la lista.

Problema 2

Utilizando la función `dot`, escribe una función que multiplique (por la izquierda) una matriz por un vector.

1 `mulmv :: Num a => [[a]] -> [a] -> [a]`

Funciones Haskell

- `import Data.List`
`transpose :: [[a]] -> [[a]]`
Devuelve la lista de listas traspuesta (si fuera una matriz, cambiando filas por columnas).

Problema 3

Utilizando las funciones anteriores, escribe otra que multiplique dos matrices.

1 `mulmm :: Num a => [[a]] -> [[a]] -> [[a]]`

Funciones Haskell

- `map :: (a -> b) -> [a] -> [b]`
Devuelve la lista resultante de aplicar la función a cada uno de los elementos de la lista.
- `sum :: Num a => [a] -> a`
Devuelve la suma de todos los elementos de la lista.
- `(**) :: Floating a => a -> a -> a`
Potenciación con números reales.
- `(/) :: Fractional a => a -> a -> a`
División de números reales.
- `fromIntegral :: (Integral a, Num b) => a -> b`
Conversión de números enteros al tipo real adecuado.

Problema 4

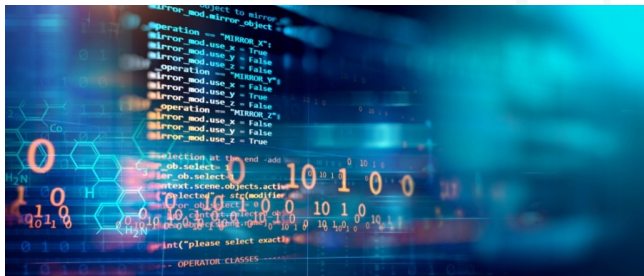
Define una función de Haskell que calcule la *norma-p* (de orden arbitrario) de una lista de números.

$$\|x\|_p = \left(\sum |x_i|^p \right)^{1/p}$$

1 `pnorm :: Floating a => Int -> [a] -> a`

Problemas de Programación Funcional

Tecnología de Programación



Adolfo Muñoz – Juan Magallón
Grado en Ingeniería Informática



Universidad
Zaragoza



Escuela de
Ingeniería y Arquitectura
Universidad Zaragoza



Departamento de
Informática e Ingeniería
de Sistemas
Universidad Zaragoza